



INTRODUCCIÓN AL DESARROLLO WEB CON DJANGO

LABORATORIO DE PROGRAMACIÓN Y LENGUAJES 2025

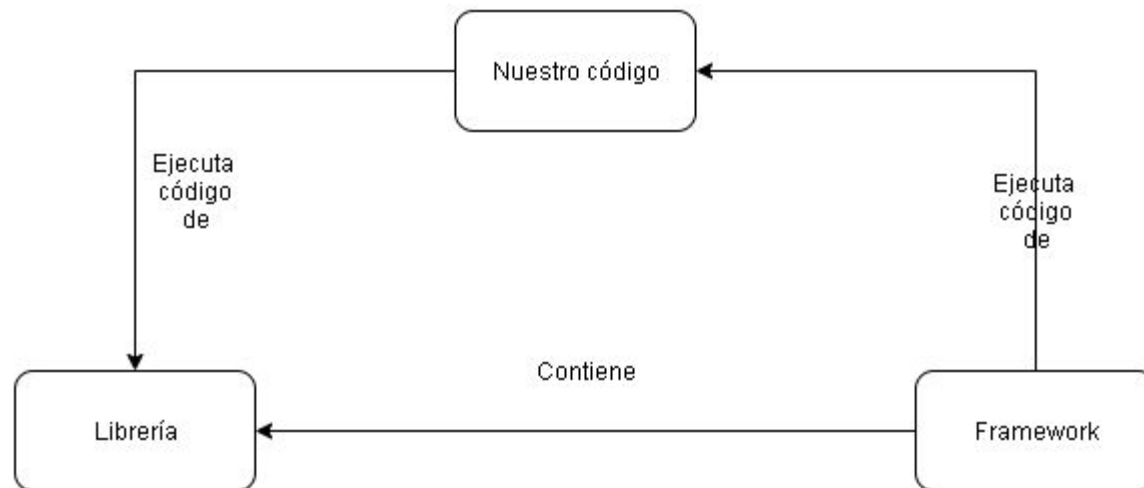


FRAMEWORK

- Wikipedia: En el desarrollo de software, un marco de trabajo es una estructura conceptual y tecnológica de asistencia definida, normalmente, con artefactos o módulos concretos de software, que puede servir de base para la organización y desarrollo de software. Típicamente, puede incluir soporte de programas, bibliotecas, y un lenguaje interpretado, entre otras herramientas, para así ayudar a desarrollar y unir los diferentes componentes de un proyecto.
- Es un conjunto de **buenas prácticas, patrones y librerías** organizados en **componentes** que permiten el desarrollo de aplicaciones para un fin específico.
- **Promueve el reuso de código.** El desarrollo de un framework requiere un esfuerzo considerable y suele requerir varias iteraciones. Escribir una aplicación de cero en un dominio en que podemos utilizar algún framework existente es similar a “re-inventar la rueda” .

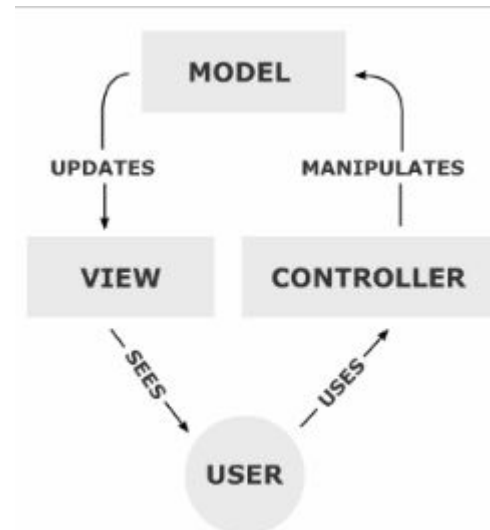
FRAMEWORK VS LIBRERÍA

- Una librería también puede contener buenas prácticas, patrones y otras librerías.
- La principal diferencia es **quien llama a quien**: En una librería es nuestro código el encargado de ejecutar porciones de la librería. Al desarrollar utilizando un framework extendemos puntos de extensión y el framework es quien se encarga de seguir el hilo de ejecución.



PATRÓN MVC –MODELO VISTA CONTROLADOR

- Es un patrón de arquitectura: **Define la forma de distribución de los principales componentes de nuestra aplicación.**
- Promueve la separación entre el **modelo de negocio, lógica de negocio y la presentación al usuario.**

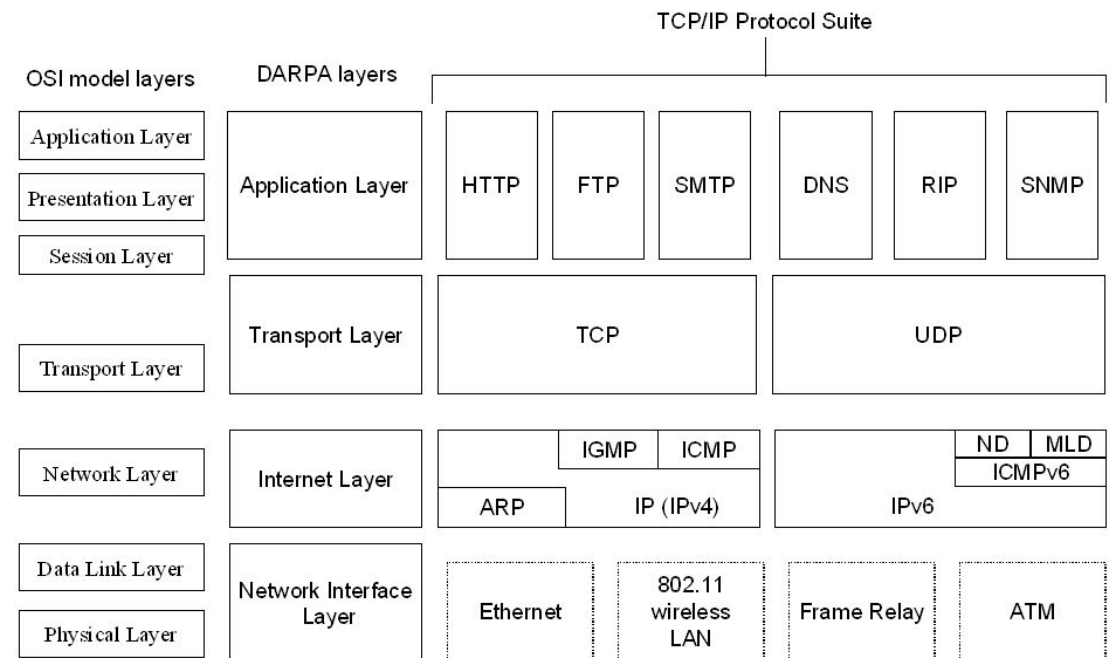


PATRÓN MVC –MODELO VISTA CONTROLADOR

- Modelo: Contiene el **modelo de dominio, los datos y la lógica de negocio** (el comportamiento).
- Vista: Contiene la **presentación al usuario**. Por ejemplo, en el caso de una aplicación web el código html (entre otras cosas) en conjunto con información que recibe del controlador.
- Controlador: Es el pegamento entre el modelo y la vista. Se encarga de recibir las entradas del usuario validarlas, ejecutar funcionalidades del modelo y retornar los resultados a la vista para ser presentados.

INTERNET

- Lo que entendemos por internet es en realidad un conjunto de protocolos a nivel de aplicación dentro de la familia TCP/IP.
- Ftp: transporte de archivos
- Http: Protocolo de transferencia de hipertexto.
- Pop: correo electrónico.
- Dns: resolución de nombres.
- ...

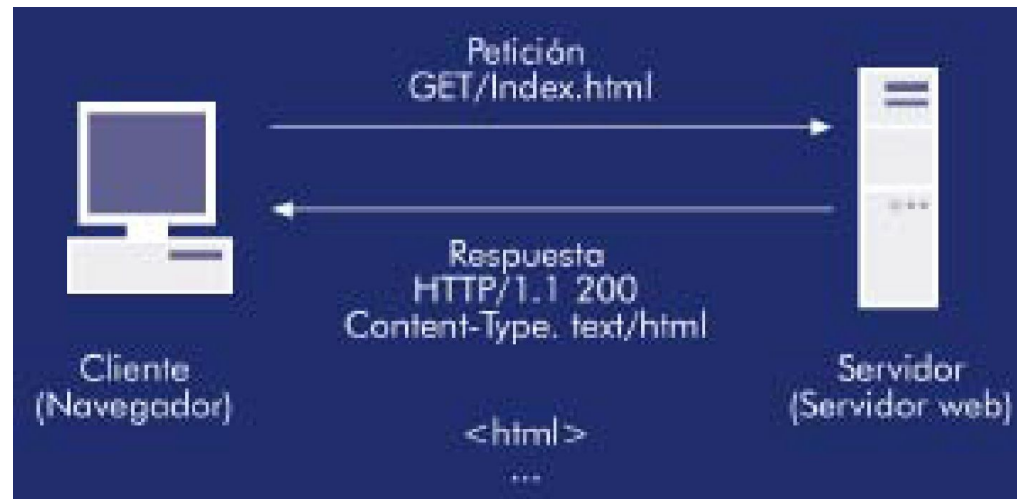


INTERNET - WEB

- El éxito de la web se basa en dos puntales fundamentales: el protocolo de comunicaciones HTTP y el lenguaje HTML.
 - HTTP: Implementación sencilla de un sistema de comunicaciones que permite enviar cualquier tipo de archivos de una forma fácil desde un navegador a un servidor, permitiendo que servidores poco potentes atiendan miles de peticiones.
 - HTML: Proporciona un mecanismo de composición de páginas enlazadas simple y fácil, altamente eficiente y de uso muy simple

INTERNET - WEB.

- Protocolo HTTP:
 - Protocolo de comunicación entre navegador y servidor web.
 - Modelo Petición/Respuesta



- Tipos de peticiones
 - GET: petición de un recurso
 - POST: envío de datos a un recurso
 - ...

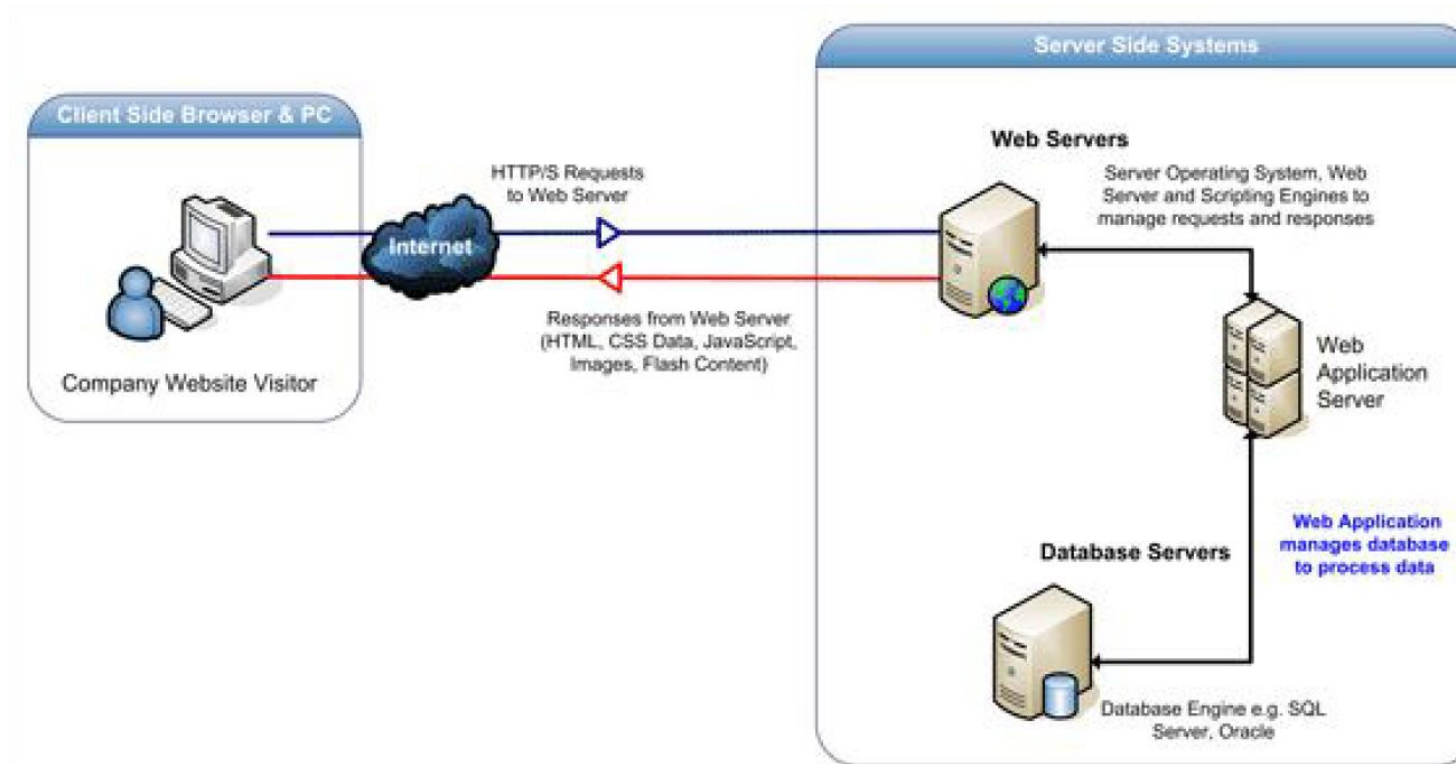
APLICACIONES WEB – SITIO WEB

- Aplicación web: aquellas aplicaciones que los usuarios pueden utilizar *accediendo a un servidor web* a través de Internet o de una intranet mediante un navegador.
 - Sitio web donde las entradas del usuario (navegación y entrada de datos) afectan al estado de un sistema de información
- Servidor web: programa que atiende y responde a las diversas peticiones de los navegadores, proporcionándoles los recursos que solicitan mediante el protocolo HTTP.

APLICACIONES WEB. CARACTERÍSTICAS

- Sistema de Información donde una **gran cantidad de datos volátiles, altamente estructurados**, van a ser consultados, procesados y analizados mediante navegadores.
- Una de las principales características es **su alto grado de interacción con el usuario**
- El **diseño de su interfaz** debe ser claro, simple y debe estar estructurado de tal manera que sea orientativo para cada tipo de usuarios

FUNCIONAMIENTO DE APLICACIONES WEB

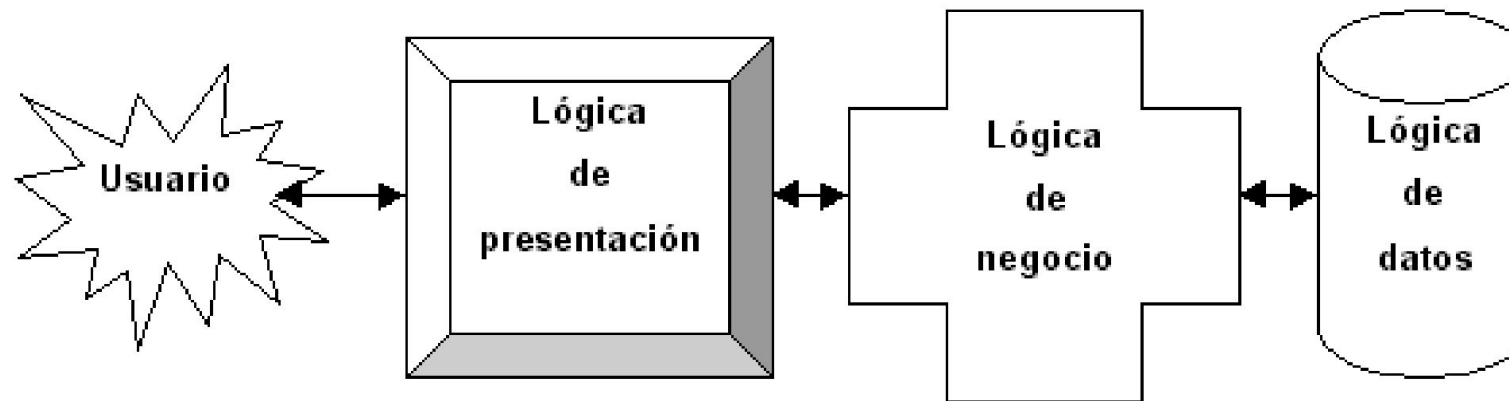


ARQUITECTURA CLIENTE/SERVIDOR

- Las aplicaciones web son un tipo especial de aplicaciones cliente/servidor. (Antes de aprender a programar aplicaciones web conviene conocer las características básicas de las arquitecturas cliente/servidor).
- Esta arquitectura implica la existencia de una relación entre procesos que solicitan servicios (clientes) y procesos que responden a estos servicios (servidores).
- La principal ventaja de esta arquitectura es que facilita la separación de las funciones según su servicio, permitiendo situar cada función en la plataforma más adecuada para su ejecución.

ARQUITECTURA CLIENTE/SERVIDOR

- **Separación de funciones** una forma frecuente de separación de aplicaciones es el modelo de tres capas.



FRAMEWORK DE DESARROLLO WEB EN PYTHON

- Un framework de desarrollo web contiene patrones y librerías organizados en componentes que suelen dar soporte a las siguientes tareas:
 - URL Routing: Realiza un vinculación entre las peticiones http y el código python a ejecutar.
 - Objetos Request(petición) y Response (respuesta): Son objetos que contienen las entradas del usuario y la respuesta que se da al usuario.
 - Motor de templatesTemplate: Contienen código html, javascript en conjunto con instrucciones propias del framework que dan vida a las vista del usuario.
 - Servidor de desarrollo: Los frameworks suelen contener un servidor de desarrollo que automáticamente levanta cualquier archivo modificado permitiendo el desarrollo rápido de aplicaciones.

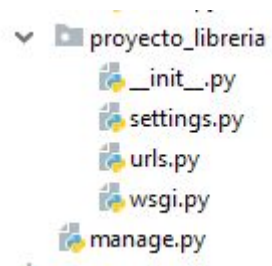
DJANGO

- Django es un framework de desarrollo web que implementa el patrón Modelo Vista Controlador completo. Es un framework maduro con años de desarrollo. Implementa pautas y patrones que permiten desarrollar aplicaciones seguras, escalables, mantenibles, y portable. Existen un sinfín de casos que prueban las cualidades de django, algunas web ejmplos son: Instagram, Youtube, Spotify, Dropbox, Mozilla, Prezy.
- La forma más fácil de instalar django es usar el gestor de paquetes python PyPip, con la instrucción “pip install django”.
- Una vez instalado tenemos disponible el script django-admin que nos va a servir como punto de partida para generar un proyecto.

DJANGO - PROYECTOS - ESTRUCTURA

- Django tiene los conceptos de proyecto y aplicaciones. Cada proyecto puede contener una o más aplicaciones. Para crear un proyecto nuevo ejecutamos:

```
es>  
es>django-admin startproject proyecto_libreria
```

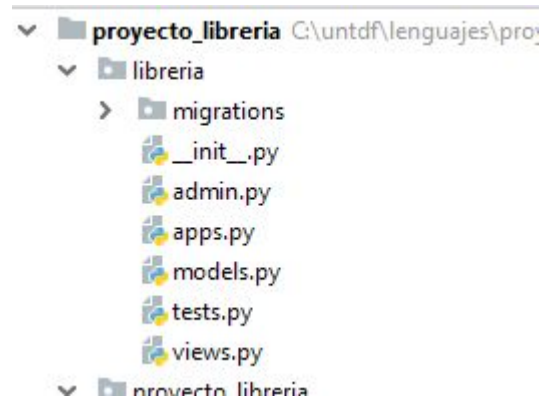


- Manage.py contiene scripts que nos permiten crear artefactos y

DJANGO – APLICACIONES -

- Para crear una aplicación utilizamos el script manager.py:

```
>  
>python manage.py startapp libreria
```

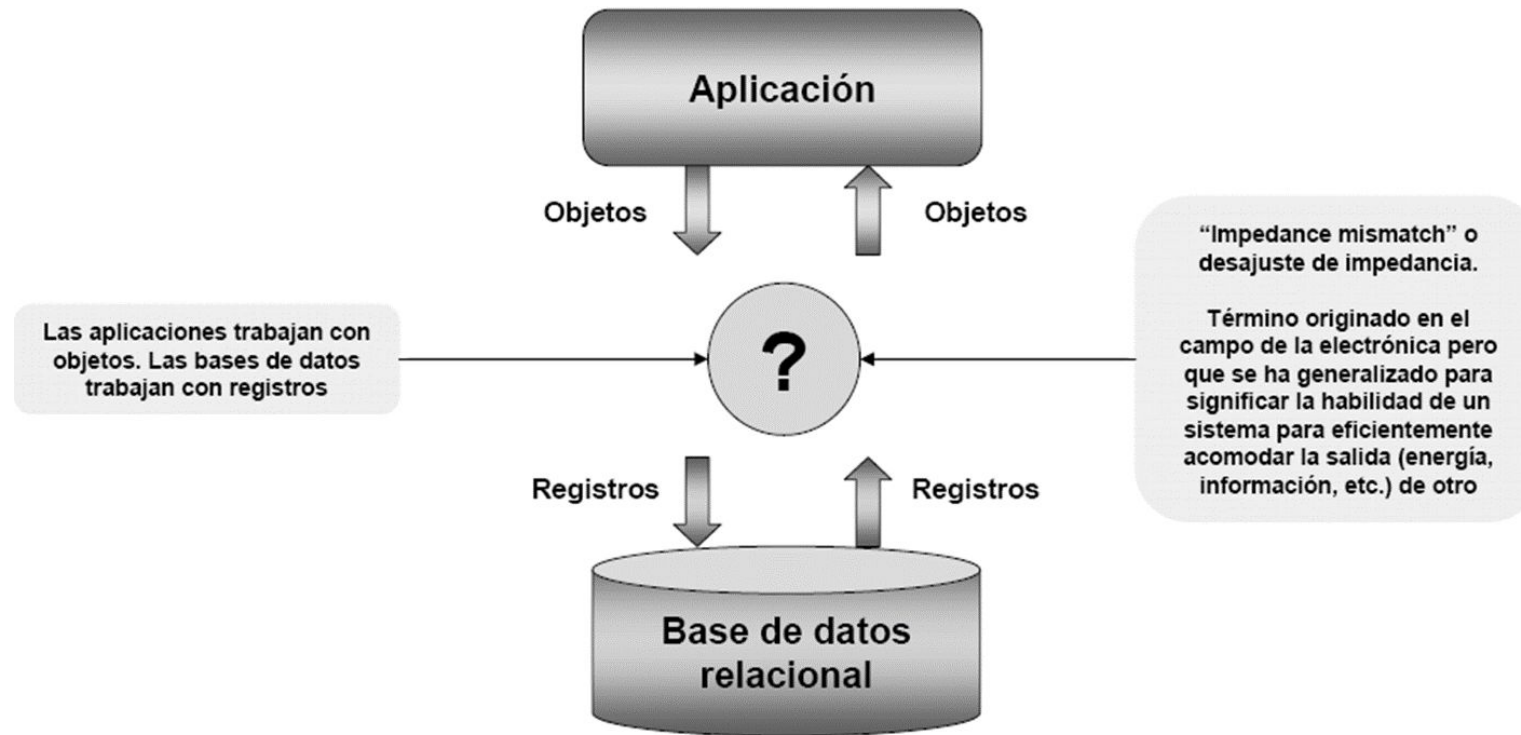


- Los scripts creados nos permiten manejar las vistas, controladores, modelos, rutas y testear la aplicación.

PERSISTENCIA DE DATOS EN ORMS

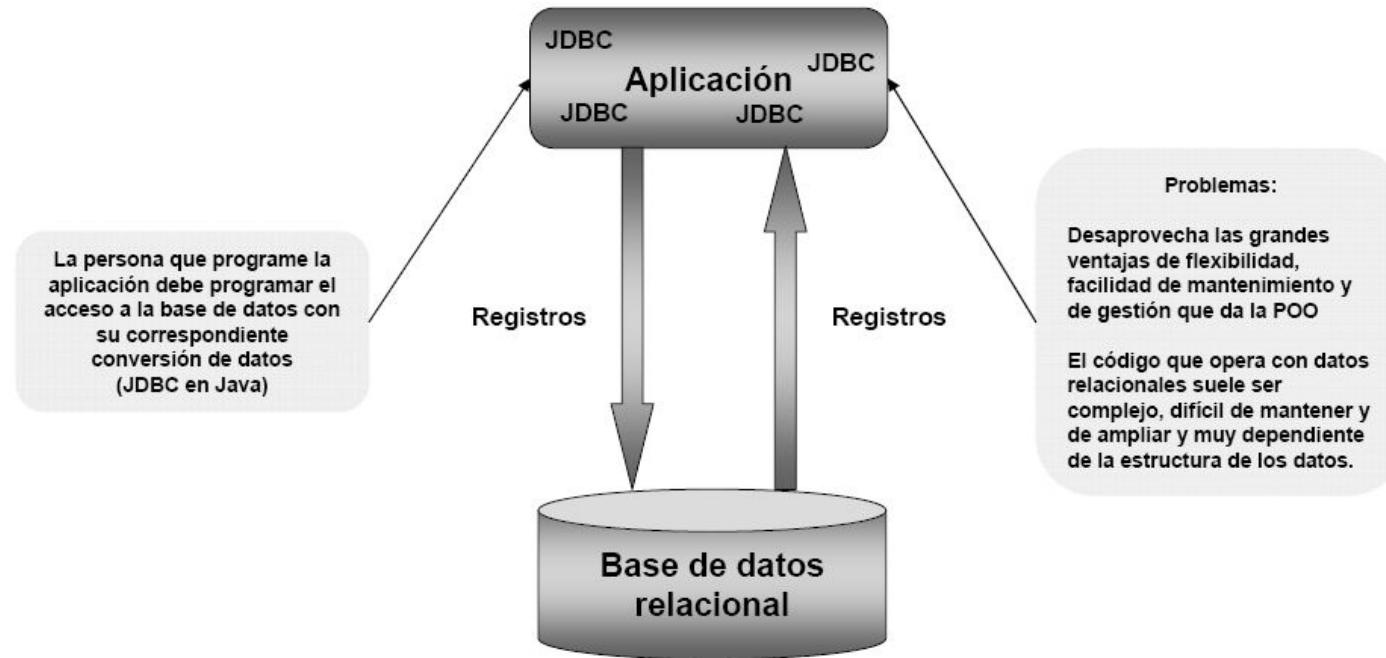
- La persistencia se define en computación, como la capacidad de los datos de sobrevivir a la aplicación que los creó.
- Se consigue almacenando los datos en archivos o bases de datos.
- La forma más sencilla de hacer persistir un objeto es serializar en archivos como por ejemplo hace Java con la interfaz **serialize**.
- Para aplicaciones que manejen gran cantidad de información interrelacionada, resulta fundamental la utilización de **Bases de datos**.

MAPEO OBJETO RELACIONAL – DIFERENCIA DE IMPEDANCIA



IMPEDANCE MISMATCH

- Solución 1: usar datos relacionales en la aplicación



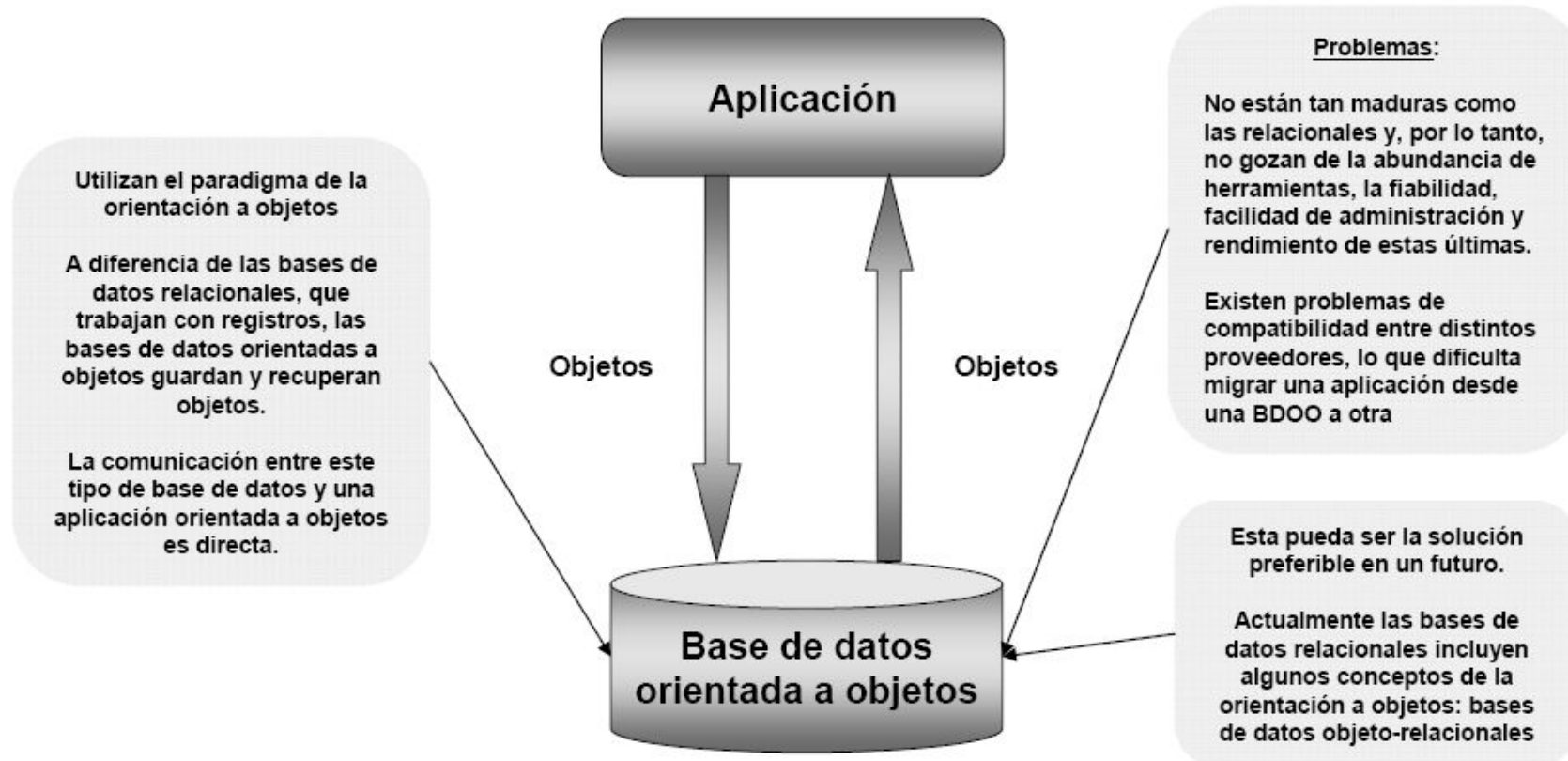
IMPEDANCE MISMATCH

- **Solución 2: usar datos relacionales en la aplicación usando un DAO (Database Access Object)**



IMPEDANCE MISMATCH

- **Solución 3: usar una base de datos orientada a objetos**



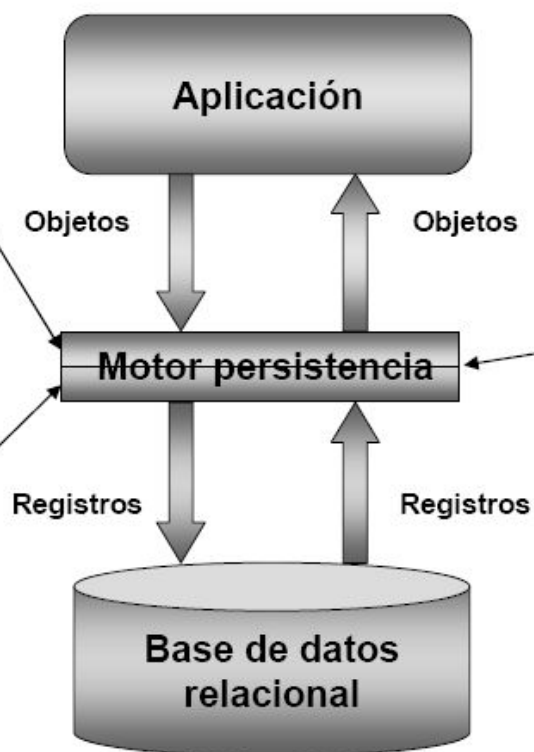
IMPEDANCE MISMATCH

- **Solución 4: usar un motor de persistencia**

Un motor de persistencia es una capa software que transforma, de forma transparente, los objetos de la aplicación en registros de una base de datos relacional.

También recibe el nombre de "Framework de persistencia" o "Mapeador Objeto-Relacional".

Cuando la aplicación quiere grabar un objeto llama al motor de persistencia, que traduce el objeto a registros y llama a la base de datos para que guarde estos registros. De la misma manera, cuando la aplicación quiere recuperar un objeto, la base de datos recupera los registros correspondientes, los cuales son traducidos en formato de objeto por el motor de persistencia.



Ventajas:

Aprovecha las ventajas de los dos modelos.

Podemos programar en POO aprovechando sus ventajas de flexibilidad, mantenimiento y reusabilidad.

Podemos usar una BD relacional, aprovechándonos de su madurez y su estandarización así como de las herramientas relacionales que hay para ella.

Con un motor de persistencia reducimos considerablemente el código, por lo tanto el desarrollo de la aplicación es menos costoso y su mantenimiento es más sencillo.

DJANGO ORM

- Django contiene un ORM que permite tanto definir nuestros modelos como realizar ABM y queries.
- El archivo models.py contiene la definición de nuestros modelos. Para que una clase pueda ser mapeada a una BD debe heredar de models.Model.
- Dentro de la clase sigue la definición de las propiedades que queremos guardar de la clase.

```
from django.db import models

class Person(models.Model):
    first_name = models.CharField(max_length=30)
    last_name = models.CharField(max_length=30)
```

```
CREATE TABLE myapp_person (
    "id" serial NOT NULL PRIMARY KEY,
    "first_name" varchar(30) NOT NULL,
    "last_name" varchar(30) NOT NULL
);
```


DJANGO ORM -CAMPOS

- Django, dentro de cada modelo tenemos campos que son los atributos de nuestras clases mapeados a campos de la base de datos.
- Django tiene funciones que permiten crear estos campos según los tipos soportados por las BD (INTEGER, VARCHAR, TEXT,.....). Django se encarga de mapear cada modelo a una tabla cuyos campos serán los atributos de la clase. El módulo de Django que se encarga de mapeos es models

```
from django.db import models

class Musician(models.Model):
    first_name = models.CharField(max_length=50)
    last_name = models.CharField(max_length=50)
    instrument = models.CharField(max_length=100)
```

- Cada campo tiene una serie de atributos que modifican y restringen los atributos posibles.

DJANGO ORM –CAMPOS - ATRIBUTOS

- Null: si es True django permitirá persistir atributos nulos en la BD.
- Blank: si es True permite persistir atributos en blanco, por ejemplo el string "". Esto es diferente a guardar un Null.
- Choices recibe una lista de tuplas con las opciones que permite el campo. Esto es similar a un tipo select de html.
- Default: permite definir un valor default.
- Help_text: permite definir un texto ayuda que será mostrado en la vista al usuario.
- primaryKey: si es True el campo es el primario del modelo.
- Unique: si es True el campo va a ser único en la tabla.